Sistema de Moeda Estudantil

Documento tutorial

1. **BACK-END**

Linguagem escolhida: Python

Banco de dados: SQL Lite

Framework: Flask
https://flask-palletsprojects-com.translate.goog/en/stable/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

O Flask é um framework do Python utilizado para criar aplicações web de forma simples. Ele pode ser usado junto com o SQLite, um banco de dados que armazena informações em um arquivo. Enquanto o Flask cuida do funcionamento do sistema, o SQLite guarda os dados, como nomes, senhas e cadastros. Essa combinação é fácil de usar e ideal para projetos pequenos e para quem está aprendendo a desenvolver sistemas.

1. Rodar o back

comandos a serem executados:

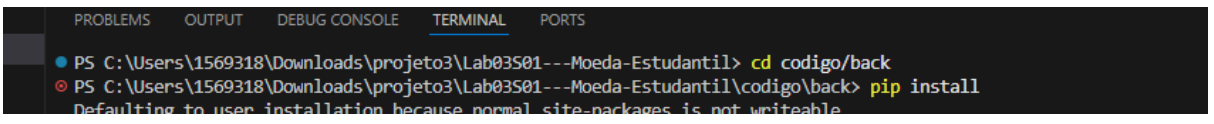
Caso não tenha estudo/trabalhado com python é necessario a instalação do interpretador em https://www.python.org/ (versão 3.11 ou superior)

1. cd codigo/back
   entra na pasta do projeto do back end para realizar os comandos

2. pip install --requirement requirements.txt
   instala as dependências usadas no projeto

3. python.exe -m pip install --upgrade pip
   caso o comando da instalação apresente algum erro, tem uma boa chance de ser um pip desatualizado

4. py run.py
   executa o arquivo responsável por executar a API



```python
"""Fábrica da aplicação Flask (arquitetura MVC).

Monta a app, inicializa o banco (SQLite), libera CORS para o front em
dev,
registra os blueprints (alunos, empresas e instituições) e semeia as
instituições de ensino pré-cadastradas quando a base ainda está vazia.
"""
import os

from dotenv import load_dotenv
from flask import Flask
from flask_cors import CORS

from app.extensions import db

load_dotenv(os.path.join(os.path.dirname(os.path.dirname(__file__)),
".env"))


def create_app():
    app = Flask(__name__)

    # DATABASE_URL permite apontar para outra base — os testes usam
SQLite em
    # memória para não recriar (e apagar) o banco de desenvolvimento.
    basedir = os.path.abspath(os.path.dirname(__file__))
    db_path = os.path.join(basedir, "..", "moedas.db")
    app.config["SQLALCHEMY_DATABASE_URI"] = os.environ.get(
        "DATABASE_URL", f"sqlite:///{db_path}"
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
    app.config["JSON_AS_ASCII"] = False

    db.init_app(app)

    # Libera a origem do front (Vite) em desenvolvimento. O header
Authorization
    # precisa ser aceito para o token de sessão chegar à API.
    frontend_origin = os.environ.get("FRONTEND_ORIGIN",
"http://localhost:5173")
    CORS(
        app,
```

```
        resources={r"/api/*": {"origins": frontend_origin}},
        allow_headers=["Content-Type", "Authorization"],
    )

    from app.controllers.aluno_controller import aluno_bp
    from app.controllers.empresa_controller import empresa_bp
    from app.controllers.instituicao_controller import instituicao_bp

    app.register_blueprint(aluno_bp)
    app.register_blueprint(empresa_bp)
    app.register_blueprint(instituicao_bp)

    # Cria as tabelas que ainda não existem e preserva os dados já
gravados.
    with app.app_context():
        from app import models  # noqa: F401  (registra as tabelas no
metadata)
        from app.seed import seed_data

        db.create_all()
        seed_data()

    return app
```

app/**__init__.py**
Arquivo da execução, onde é possível alterar informações da execução **incluir controllers**, os arquivos salvos na pasta models são lido **sem a necessidade de incluir novas linhas**

2. Models

Explicação do model de Usuário/Aluno

```python
"""Model dos alunos (cadastro, saldo de moedas e instituição de ensino).

O aluno é um ``Usuario``: herda ``login`` (o e-mail) e ``senha_hash``. O ``saldo`` é
sempre alterado por transações (recebimento de moedas e troca por vantagem),
nunca direto pela tela de cadastro.
"""
from app.extensions import db
from app.models.usuario import Usuario


class Aluno(Usuario):
    __tablename__ = "alunos"

    id = db.Column(db.Integer, db.ForeignKey("usuarios.id"), primary_key=True)

    nome = db.Column(db.String(120), nullable=False)
    email = db.Column(db.String(160), nullable=False, unique=True)
    cpf = db.Column(db.String(11), nullable=False, unique=True)  # só dígitos
    rg = db.Column(db.String(20), nullable=False)
    endereco = db.Column(db.String(255), nullable=False)
    curso = db.Column(db.String(120), nullable=False)
    saldo = db.Column(db.Integer, nullable=False, default=0)
```

Atributos da tabela de Alunos, Db.collum é usado para definir as colunas no banco, recebendo parâmetros conforme necessário, obrigatório: tipo

```python
    instituicao_id = db.Column(
        db.Integer, db.ForeignKey("instituicoes_ensino.id"), nullable=False
    )
    instituicao = db.relationship("InstituicaoEnsino", back_populates="alunos")

    __mapper_args__ = {"polymorphic_identity": "aluno"}

    def receber_moedas(self, valor):
        self.saldo += valor

    def debitar(self, valor):
        if valor > self.saldo:
            raise ValueError("Saldo insuficiente")
        self.saldo -= valor

    def to_dict(self):
        return {
            "id": self.id,
            "nome": self.nome,
            "email": self.email,
            "cpf": self.cpf,
            "rg": self.rg,
            "endereco": self.endereco,
            "curso": self.curso,
            "saldo": self.saldo,
            "instituicaoId": self.instituicao_id,
            "instituicaoNome": self.instituicao.nome,
        }
```

Chaves estrangeiras, métodos, e o to_dict usado para transformar a classe em json com mais facilidade

2. Controller

```python
aluno_bp = Blueprint("alunos", __name__, url_prefix="/api/alunos")
```

Variável importante para a criação e execução do controller, ela quem é lida no init

```python
def _erro(mensagem, status, campos=None):
    corpo = {"erro": mensagem}
    if campos:
        corpo["campos"] = campos
    return jsonify(corpo), status
```

Metodo facilitador para devolver a requisição com erros

```python
@aluno_bp.post("")
def criar():
    dados = request.get_json(silent=True)
    if not isinstance(dados, dict):
        return _erro("Envie os dados em JSON.", 400)

    campos, instituicao = _validar(dados, CAMPOS_ALUNO + ["senha"])
    if campos:
        return _erro("Dados inválidos", 400, campos)

    email = texto(dados, "email").lower()
    cpf = apenas_digitos(texto(dados, "cpf"))

    # O e-mail é o login, então não pode se repetir em nenhum perfil.
    if Usuario.query.filter_by(login=email).first():
        return _erro("E-mail já cadastrado", 409)

    if Aluno.query.filter_by(cpf=cpf).first():
        return _erro("CPF já cadastrado", 409)

    aluno = Aluno(
        login=email,
        nome=texto(dados, "nome"),
        email=email,
        cpf=cpf,
        rg=texto(dados, "rg"),
        endereco=texto(dados, "endereco"),
        curso=texto(dados, "curso"),
        instituicao_id=instituicao.id,
        saldo=0,
    )
    aluno.definir_senha(dados["senha"])
    db.session.add(aluno)

    try:
        db.session.commit()
    except IntegrityError:
        db.session.rollback()
        return _erro("E-mail ou CPF já cadastrado", 409)

    return jsonify(aluno.to_dict()), 201
```

Método da criação de um aluno, db.session.add/commit que é responsável pela persistência do dado

dados do corpo da requisição são lidos com[

```
dados = request.get_json(silent=True)
dados.get(“nome”)
```

assim eu teria acesso ao dado digitado em
{
…
“nome”: “testes”,
…
}

Buscar

```
@aluno_bp.get("")
def listar():
    alunos = Aluno.query.order_by(Aluno.nome).all()
    return jsonify([aluno.to_dict() for aluno in alunos])
```

Buscas são realizadas a partir do objetivo do model, possuindo metodos como

- filter(): permite buscar registros que atendam a determinadas condições, como encontrar alunos de um curso específico.
- limit(): limita a quantidade de registros retornados pela consulta.
- order_by(): organiza os resultados em uma ordem específica, como por nome.
- first(): retorna apenas o primeiro registro encontrado.
- all(): retorna todos os registros encontrados pela consulta.

```
@aluno_bp.put("/<int:aluno_id>")
def atualizar(aluno_id):
    aluno = _buscar(aluno_id)

    if aluno is None:
        return _erro("Aluno não encontrado", 404)

    dados = request.get_json(silent=True)
    if not isinstance(dados, dict):
```

```
        return _erro("Envie os dados em JSON.", 400)

    campos, instituicao = _validar(dados, CAMPOS_ALUNO)
    if campos:
        return _erro("Dados inválidos", 400, campos)

    email = texto(dados, "email").lower()
    cpf = apenas_digitos(texto(dados, "cpf"))

    outro_email = Usuario.query.filter(
        Usuario.login == email,
        Usuario.id != aluno_id,
    ).first()

    if outro_email:
        return _erro("E-mail já cadastrado", 409)

    outro_cpf = Aluno.query.filter(
        Aluno.cpf == cpf,
        Aluno.id != aluno_id,
    ).first()

    if outro_cpf:
        return _erro("CPF já cadastrado", 409)

    aluno.nome = texto(dados, "nome")
    aluno.email = email
    aluno.login = email
    aluno.cpf = cpf
    aluno.rg = texto(dados, "rg")
    aluno.endereco = texto(dados, "endereco")
    aluno.curso = texto(dados, "curso")
    aluno.instituicao_id = instituicao.id

    # Senha é opcional na edição: só troca se vier preenchida.
    senha = dados.get("senha")
    if senha:
        aluno.definir_senha(senha)

    try:
        db.session.commit()
    except IntegrityError:
        db.session.rollback()
```

```
        return _erro("E-mail ou CPF já cadastrado", 409)

    return jsonify(aluno.to_dict())
```

Edição de um dado, busca o dado pelo model, faz as alterações como no exemplo, e lança db.session.commit() para persistir o dado

```
@aluno_bp.delete("/<int:aluno_id>")
def remover(aluno_id):
    aluno = _buscar(aluno_id)

    if aluno is None:
        return _erro("Aluno não encontrado", 404)

    try:
        db.session.delete(aluno)
        db.session.commit()
    except IntegrityError:
        db.session.rollback()
        return _erro(
            "Não foi possível remover o aluno. "
            "Verifique se existem registros vinculados.",
            409,
        )

    return "", 204
```

Para remover busca o aluno pelo model, e lança db.session.delete

2. FRONT END

O front nós usamos o React com css puro
https://react.dev/

2. Rodar o front

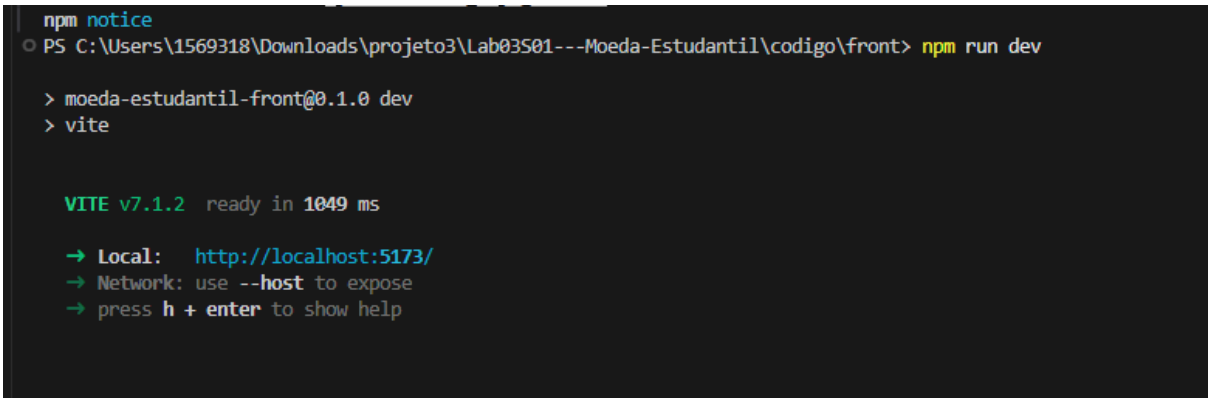
comandos a serem executados:

Caso não tenha estudo/trabalhado com JS ou react é necessario a instalação do interpretador em https://nodejs.org/pt-br

1. cd codigo/front
   entra na pasta do projeto do front-end para realizar os comandos

2. npm i
   o npm i é responsavel por instalar as dependencias em package.json

3. npm run dev
   O npm run dev é utilizado para iniciar o projeto em modo de desenvolvimento
   resultado esperado:




```jsx
import { useState } from "react";

import Alunos from "./pages/Alunos.jsx";
import Empresas from "./pages/Empresas.jsx";

const ABAS = [
  { id: "alunos", rotulo: "Alunos", Pagina: Alunos },
  { id: "empresas", rotulo: "Empresas parceiras", Pagina: Empresas },
];

export default function App() {
  const [abaAtual, setAbaAtual] = useState(ABAS[0].id);
  const { Pagina } = ABAS.find((aba) => aba.id === abaAtual);
```

```
  return (
    <>
      <header className="topo">
        <h1>Moeda Estudantil</h1>
        <nav>
          {ABAS.map((aba) => (
            <button
              key={aba.id}
              type="button"
              className={aba.id === abaAtual ? "aba ativa" : "aba"}
              onClick={() => setAbaAtual(aba.id)}
            >
              {aba.rotulo}
            </button>
          ))}
        </nav>
      </header>

      <main>
        <Pagina />
      </main>
    </>
  );
}
```

App.jsx quem é a responsavel por definir a navegação entre as telas